



Revision Notes on HashMaps and HashSets

Introduction

In this session, we explored the concepts of HashMaps and HashSets, focusing on their structure, practical applications, and various use cases. HashMaps and HashSets are fundamental data structures used for storing and managing collections of unique items and key-value pairs.

HashMap

A HashMap is a data structure that stores elements in a key-value pair format. It allows for fast access, insertion, deletion, and searching of elements. The time complexity for these operations is $O(1)$ due to its hashing mechanism.

Example Use Case: Consider a hotel with several rooms needing a system to track occupancy status. Initially, this data can be tracked using a HashMap, where the room number is the key, and whether it's occupied is the value.

Advanced Use Case:

Imagine a scenario where room numbers were changed to random "lucky" numbers between 1 and 10^9 due to a belief system. Maintaining a status-tracking array of that size is inefficient. Instead, a HashMap is used to associate each room's status with its new number `【4:18+transcript.txt】`.

HashSet

A HashSet is a collection that stores unique elements. There is no duplicate storage, and elements cannot have a key-value pair association like HashMaps. Like HashMaps, the time complexity for insertions, deletions, and queries is $O(1)$.



- **HashMap:** Stores elements in key-value pairs.
- **HashSet:** Stores only unique keys without any associated values
【4:18+transcript.txt】 .

Frequency Counting using HashMap

We tackled a frequency counting problem using HashMaps. This demonstrates how HashMaps can efficiently count elements' frequency in a given array:

1. **Brute-force Approach:** Iterate over each element and check its count in a nested loop. This results in a time complexity of $O(N^2)$.
2. **Optimized Approach:** Using a HashMap to store each element's count, reducing the time complexity to $O(N)$.

Pseudocode Explanation:

Here is the pseudocode for counting the frequency of elements using a HashMap:

```
function frequencyCounter(array):  
    hashmap = {}  
    for element in array:  
        if element in hashmap:  
            hashmap[element] += 1  
        else:  
            hashmap[element] = 1  
    return hashmap
```

This approach efficiently solves the problem with a linear time complexity of $O(N)$ 【4:18+transcript.txt】 .

Distinct Elements Problem using HashSet

Another problem we addressed was determining the number of distinct elements in an array. The task is effectively solved using a HashSet:



- The size of the HashSet at the end of the process gives the number of distinct elements 【4:18+transcript.txt】 .

Pseudocode for Counting Distinct Elements:

```
function countDistinctElements(array):
    hashset = set()
    for element in array:
        hashset.add(element)
    return len(hashset)
```

Optimized Pair Sum Problem using HashMap

To find pairs in an array that add up to a particular sum (target), we use a HashMap to track elements we have encountered:

1. For each element, compute the complement (target - current element).
2. Check if this complement exists in the HashMap.
3. If it exists, increment the count of pairs.
4. Update the HashMap with the current element 【4:19+typed.md】 .

Pseudocode:

```
function countTargetSum(arr[], K):
    N = arr.length;
    Hashmap<integer, integer> hm;
    count = 0;
    for i from 0 to N-1:
        target = K - arr[i]
        if target in hm:
            count += hm[target]
        if arr[i] in hm:
            hm[arr[i]] += 1
        else:
            hm[arr[i]] = 1
    return count
```

Conclusion



involving frequency counting, finding distinct elements, and solving pair sum problems. Understanding these structures and their $O(1)$ operational complexity is crucial for optimizing algorithms that deal with large datasets 【4:17+transcript.txt】 【4:15+transcript.txt】 .